CIS634 Project — Meeting Cheat Sheet

Nov 1, 2026 · @Jacob

The 30-second pitch

Say this first: "We tested whether off-the-shelf sentence embeddings can find NASA aviation safety reports that describe the same problem in different words. Surprisingly, plain keyword search (BM25) beat every embedding model at finding a specific report. Combining the two works best for topic search. Embeddings also get fooled by small edits that flip the meaning when the words stay the same."

- **Project:** Semantic Retrieval of Aviation Safety Reports with Frozen Sentence Embeddings (CIS 634, Dr. Mansi Bhavsar). Jacob Tang and Aarambh Shrestha.
- **Research question:** Can a frozen (not fine-tuned) sentence embedding retrieve ASRS reports about the same issue when the wording differs? Does it fail on negation, numbers, direction or event order?
- **Project type:** an application plus an analysis project. We train no model; we benchmark and analyse existing ones.
- **Three headline numbers:** BM25 finds the right report first 36% of the time vs. 21% for MiniLM. BM25 + MiniLM fusion gives the best topic ranking (nDCG@10 0.706 vs. 0.630). With word overlap controlled, E5 catches meaning changes 85% of the time, but event order stays near chance.

The experiment in plain words

We index all 47,723 public ASRS reports, run five search systems on them, and score each one on three tests.

Data. The Hugging Face dataset `elihoole/asrs-aviation-reports` (Apache-2.0): 47,723 de-identified pilot and controller incident reports, Jan 2012 to Mar 2022. The official split is 38,655 train, 4,295 validation and 4,773 test. Each report has 1 or 2 narratives (319 words on average), a 27-word analyst synopsis, and NASA analyst codes. **We search only the narratives.**

Systems compared.

System	What it is, in one line	Size
BM25	Classic keyword search: rewards rare shared words. We wrote it ourselves.	no model
MiniLM (all-MiniLM-L6-v2)	Sentence encoder: turns text into a 384-number vector; similar meaning = close vectors	22M params
E5-small (e5-small-v2)	A newer encoder trained for search; needs "query: " / "passage: " prefixes	33M params
BM25 + E5 (RRF)	Merges the two rankings with reciprocal rank fusion	–
BM25 + MiniLM (RRF)	Same, with MiniLM	–

The extended run adds BGE-small (33M), MPNet (110M) and E5-base (110M), plus ablations.

Long reports. Reports are longer than the encoders can read, so we cut each into roughly 150-word chunks. A report's score is its best chunk's score (called MaxP). The best chunk is also shown to the user as the matched passage.

The three tests ("tracks").

Track	Query	What counts as correct	Metric
Synopsis (automatic)	The analyst's synopsis of 1,000 validation reports	The report it summarises, or another report of the same incident	MRR@10, Recall@1/10/100
Topical	40 issue descriptions we wrote (20 dev, 20 test), across 15 event types	Graded 2 = same issue, 1 = related, 0 = not. "Silver" labels come from NASA's codes	nDCG@10, P@10
Contrast (meaning change)	100 hand-written items: 25 each of negation, numbers, direction, event order	The model rates the same-meaning paraphrase closer than the edited, meaning-changed sentence	Accuracy (chance = 0.50)

Contrast example. Original: "The crew did not descend." Paraphrase: "The crew maintained altitude." Edit: "The crew descended." A good model should say the paraphrase is closer.

Statistics. Every mean has a 95% bootstrap confidence interval. Pairs of systems are compared with a paired randomization test (p-values).

Steps taken, in order

We built the pipeline in eight steps. Pre-processing went through two versions, and that change is itself a finding.

1. **Proposal (Phase 1).** Picked the question, the dataset and the four systems: BM25, MiniLM, E5, and hybrids.
2. **Data preparation** (`asrsbench prepare`):
   a. *Comma repair.* The export had turned every comma into a semicolon ("10;000 FT"). We restored commas in the free text so numbers read correctly.
   b. *Placeholder removal.* 2,906 second narratives (26% of them) were editorial notes such as "[Report narrative contained no additional information.]". We drop them.
   c. *Incident grouping.* Union-find links reports that share an accession number or identical text. 70 reports belong to multi-report incidents. Retrieving a sibling report counts as correct.
   d. *Chunking.* The narratives have no paragraph breaks, so we pack sentences into windows of at most 150 words with a one-sentence overlap. A leftover tail under 40 words merges into the previous window. Result: 136,842 chunks, about 2.9 per report.
3. **Retrieval layer.** One shared interface, so BM25, any sentence-transformers model and hybrids all plug in the same way. Adding a model takes one YAML entry.
4. **Evaluation harness.** The three tracks, the metrics, bootstrap confidence intervals and significance tests. We wrote the 40 topical queries with their silver-label rules, plus the 100 contrast items.
5. **First run (v1) and error analysis.** In E5's top-10 results, 18.5% of the matched passages were short fragments, mostly placeholders. That led to pre-processing v2 (placeholder removal + tail merging). The v1 results are kept in `results/milestone_v1_rawchunks/` as the "before" condition.
6. **Milestone run (v2)** of all five systems on all three tracks (`configs/milestone.yaml`).

7. **Extended run.** Three more encoders plus ablations: whole report vs. best chunk, and BM25 that keeps "not" (`configs/extended.yaml`).
8. **Fixes, analysis and review.**
   - Fixed the silver-label patterns to match at word starts, then reran everything.
   - Added `scripts/analysis.py` for the error analyses.
   - Revised the milestone report after an adversarial review (PR #1, merged).
   - Generated blind judging pools for the human labels (`data/qrels/pool_v1/`).

Key results and what they mean

BM25 wins on finding a specific report, fusion wins on topic search, and every system is fooled when an edit keeps the words. Full tables with confidence intervals: `results/milestone/summary.md` .

Main results (milestone run, 47,723 reports):

System	Synopsis MRR@10	Synopsis R@100	Topical nDCG@10	Topical P@10	Contrast (controlled)
BM25	**0.425**	0.753	0.630	0.537	0.53
MiniLM	0.268	0.648	0.591	0.463	0.70
E5-small	0.241	0.607	0.438	0.352	**0.85**
BM25 + E5 (RRF)	0.378	0.759	0.633	0.537	–
BM25 + MiniLM (RRF)	0.380	**0.786**	**0.706**	**0.585**	–

Confidence intervals are about ±0.03 for synopsis and ±0.08 for topical (only 40 queries).

How to read the numbers:

- **MRR@10 0.425** is the average of 1/rank of the correct report (counted as 0 if it's outside the top 10). BM25 puts it at rank 1 for 36% of queries (MiniLM 21%, E5 19%) and in the top 10 for 56%.
- **R@100 0.786** means the correct report is somewhere in the top 100 for 78.6% of queries.
- **nDCG@10** (0 to 1) rewards putting grade-2 reports above grade-1 ones near the top.
- **P@10 0.585** means about 6 of the top 10 results are the "same issue".

- **Contrast 0.85** means E5 picked the paraphrase over the meaning-changed edit in 85 of 100 items. 0.50 is a coin flip.

Meaning-change test by category (controlled condition: the anchor is the paraphrase, so word overlap is balanced):

System	Negation	Numbers	Direction	Event order	All
BM25	0.22	0.60	0.78	0.50	0.53
MiniLM	0.68	0.96	0.76	0.40	0.70
E5-small	0.92	0.96	0.92	0.60	0.85

In the original condition (the anchor is the original, so the edit shares almost every word with it), every system fails: accuracy is 0.05 to 0.24.

Extended run (`results/extended/summary.md`):

System	Synopsis MRR@10	Topical nDCG@10	Contrast all	Event order
BM25, whole report	**0.458**	0.621	0.53	0.50
BM25, keep negation words	0.428	0.626	0.49	0.50
BGE-small (33M)	0.287	0.610	0.85	0.60
MPNet (110M)	0.257	0.622	0.85	0.72
E5-base (110M)	0.280	0.451	0.82	0.52
BM25 + BGE (RRF)	0.393	0.658	–	–

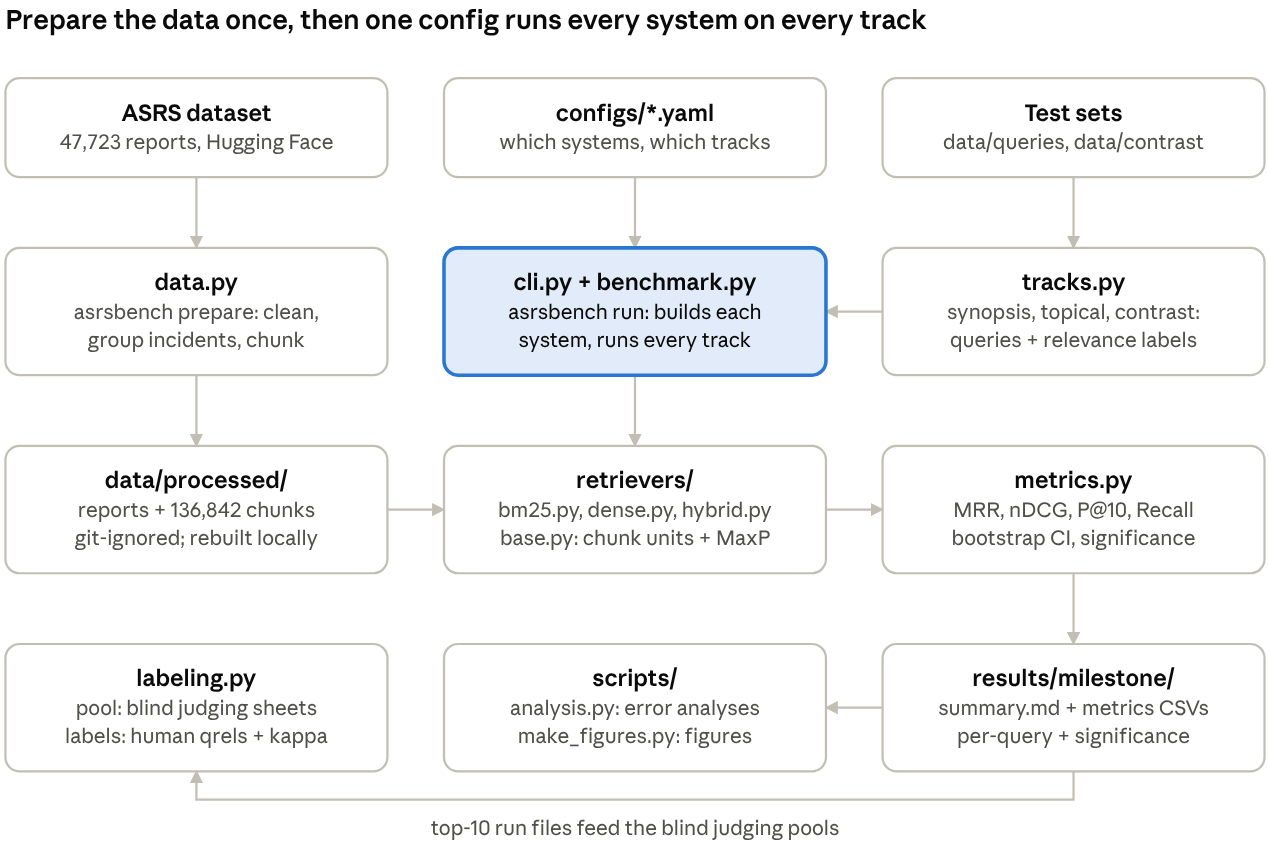
Five findings to say out loud:

1. **BM25 is the best single system on synopses, the opposite of what we expected.** It's not just shared identifiers: only 29% of synopses repeat an airport code, aircraft type or runway, and BM25 still leads on the other 71% (0.321 vs. 0.196). A likely reason is ASRS jargon (FL240, LNAV/VNAV, RA) that general-English encoders rarely see.
2. **Bigger encoders don't close the gap.** None comes within 0.13 MRR@10 of BM25 on synopses. On topical queries, BM25 vs. MiniLM, MPNet and BGE are not significantly different (BM25 vs. MiniLM p = 0.29).

3. **Fusion helps topic search** (+0.076 nDCG@10 over BM25, p = 0.0015). On synopses it ranks the target first less often than BM25 alone (MRR@10 0.380 vs. 0.425).
4. **Word overlap beats meaning.** With overlap controlled, the encoders catch most negation and number edits. Event order stays near chance. BM25 passes 0.78 of direction items, so direction can be passed by word overlap alone.
5. **E5 favours short texts, but that explains little.** Short units fill 17.5% of E5's top 10 but only 0.9% of the index. Fixing them would raise its P@10 by about 0.01, against a 0.11 gap to MiniLM.

Codebase map

The code is one Python package, `src/asrsbench/` (about 1,400 lines), driven by YAML configs. Its key design idea is that every search system shares one interface, so they all run on identical data and tests.



asrsbench pipeline · how the files connect

Read it top to bottom: inputs, then the runner (highlighted) pulls in the chunks, the systems and the test queries, scores the rankings and writes a results folder.

Path	What it does
`src/asrsbench/data.py`	Download, comma repair, placeholder removal, union-find incident grouping, sentence chunking
`retrievers/base.py`	Shared `Retriever` interface. `Units` = chunks or whole reports; MaxP = a report's score is its best unit's
`retrievers/bm25.py`	Our BM25 on sparse matrices, Snowball stemming, optional keep-negation stopword list
`retrievers/dense.py`	Any sentence-transformers model: prefixes, auto device, embedding cache
`retrievers/hybrid.py`	RRF (or weighted min-max) fusion of systems defined earlier in the config
`retrievers/__init__.py`	`build_systems`: turns YAML entries into retriever objects
`tracks.py`	Samples the synopsis queries, applies silver-label rules, loads contrast items and human qrels
`metrics.py`	P@k, Recall@k, MRR, nDCG, bootstrap CI, paired randomization test, Wilson CI
`benchmark.py`	`run_benchmark`: indexes and times each system, runs every track, writes the results folder
`labeling.py`	`build_pool` (blind sheets, 25% overlap between annotators) and `import_labels` (qrels + kappa)
`cli.py`	The `asrsbench` command: `prepare`, `run`, `search`, `pool`, `labels`
`scripts/analysis.py`	The error analyses quoted in the report: overlap, identifiers, short hits, E5 score spread

Path	What it does
`scripts/make_figures.py`	The contrast figure, per-facet tables and failure examples
`tests/`	22 pytest unit tests, including a check that our BM25 matches `bm25s`
`configs/`	`smoke.yaml` (quick check), `milestone.yaml` (main table), `extended.yaml` (more encoders + ablations)
`data/queries`, `data/contrast`, `data/qrels`	40 topical queries + label rules, 100 contrast items, judging pools
`results/`	`milestone`, `extended`, and `milestone_v1_rawchunks` (the run before pre-processing v2)
`reports/milestone/`	IEEE LaTeX report (`main.tex`, `main.pdf`) and figures
`Research/`	The proposal and the class guidelines

How to run it (and a live demo)

Everything runs through one command, `asrsbench`. A fresh clone prepares the data once, then each config runs end to end.

```
# setup (once)
python -m venv .venv && .venv/bin/pip install -e ".[dev]"
source .venv/bin/activate

asrsbench prepare                      # download + clean + chunk ->
data/processed/
asrsbench run configs/smoke.yaml       # quick end-to-end check (a few
minutes)
asrsbench run configs/milestone.yaml   # the main table: 5 systems x 3 tracks
asrsbench run configs/extended.yaml    # extra encoders + ablations
python scripts/analysis.py results/milestone   # error analyses ->
analysis.json
python scripts/make_figures.py results/milestone   # contrast figure + facet
tables
pytest                                 # 22 unit tests (all pass, ~2 s)
```

Best live demo (fast once the embeddings are cached):

```
asrsbench search configs/milestone.yaml bm25 "crew descended to the altitude
meant for another aircraft"
asrsbench search configs/milestone.yaml e5-small "crew descended to the
altitude meant for another aircraft"
```

This prints the top 5 reports with the matched passage, so you can show BM25 and an encoder side by side.

Good to know:

- The device (CUDA, Apple MPS or CPU) is picked automatically. It also runs on Google Colab with `!pip install -e .`.
- Embeddings are cached in `cache/embeddings/`, so a rerun only encodes the queries.
- `data/processed/` and `cache/` are git-ignored (too big), so `asrsbench prepare` rebuilds them from Hugging Face.
- Timing on an M1 Max: indexing takes 7 s for BM25, 2.6 min for MiniLM and 4.8 min for E5. 1,000 queries take under 3 s per system.
- Each run writes `results/<run>/`: `summary.md` (readable tables), `metrics_*.csv`, `per_query_*.csv`, `significance_*.csv` and `meta.json` (timings, environment).

Likely professor questions, with short answers

Why is this an NLP project if you train nothing? It's an application and analysis project: we compare lexical, neural and hybrid retrieval on real domain text, then probe what the encoders do and don't capture (negation, numbers, direction, order). The class guidelines list both project types.

What did you build yourselves vs. reuse? Ours: the data pipeline, the BM25 implementation, MaxP scoring and fusion, the YAML config system, the 40 queries and their labelling rules, the 100 contrast items, and all evaluation code (about 1,400 lines of Python, 22 unit tests). Reused: sentence-transformers, Hugging Face `datasets`, scikit-learn, PyStemmer and six pretrained checkpoints.

Why write your own BM25? So the same corpus statistics can also score sentence pairs for the contrast test. A unit test checks our scores match the `bm25s` library.

Isn't the synopsis track biased toward BM25? Partly. 53% of a synopsis's content words appear in its own narrative (15% in a random one), so it rewards word overlap, and we say so. But BM25 still leads on the 71% of synopses with no shared identifiers.

Why did E5 do so badly on retrieval but best on the contrast test? The two tests measure different things. On retrieval, E5 over-ranks short passages and struggles with altitude queries (nDCG@10 0.159). On isolated sentence pairs with balanced overlap, it separates meanings best (0.85).

Why two conditions in the contrast test? In the proposal's condition, the edit shares almost every word with the original, so every system is fooled (5 to 24% correct). We added a controlled condition where the anchor is the paraphrase, which overlaps both candidates equally (word Jaccard 0.29 each). That separates "fooled by overlap" from "can't encode meaning".

Are the silver labels trustworthy? They're a proxy built from NASA's analyst codes and, for 24 queries, synopsis keywords. Grade-2 sets are broad (median 725 reports per query). Human labels from blind pooled judging will check them.

Did you peek at the test data? The synopsis test split is untouched and will be scored once for the final report. The 20 topical test queries are already scored, so the new queries we write for human labelling become the held-out set.

Limitations to admit up front:

- Human relevance labels are not done yet. Pools exist (845 rows for annotator A, 819 for B, 10 queries shared for Cohen's kappa), but none are graded.

- Only 40 topical queries, so confidence intervals are wide (about ±0.08). The contrast categories have 25 items each (about ±0.19).
- The contrast items are hand-written sentences, not real report text.
- No aviation-specific encoder (Aviation-BERT, SafeAeroBERT) and no fine-tuning yet.
- The whole-report ablation also truncates long reports, so it doesn't isolate best-chunk scoring.

What's next (by the final report on Nov 30):

- [ ] Both annotators grade the pools, then `asrsbench labels ...` produces `human_v1.tsv` and kappa
- [ ] Extend toward the proposed 50 to 80 topical queries
- [ ] Test score calibration for E5's length bias, and fusion weights tuned on dev queries
- [ ] Stretch goals: report versions (B), fine-tuning on near-miss pairs split by incident group (D), ADS-B tracks (C)
- [ ] Score the synopsis test split once
- [ ] Slides for the Nov 23 presentation (6 to 8 slides)

Glossary: say it simply

Term	Plain explanation
ASRS	NASA's Aviation Safety Reporting System: voluntary, confidential incident reports from pilots, controllers and crew
Narrative / synopsis	The reporter's own free-text story / the analyst's short (about 27-word) summary of it
ACN	Accession number, the ID of one report
BM25	Keyword ranking: scores shared words, weighting rare words more and long documents less (k1 = 1.2, b = 0.75)
Stemming / stopwords	Cutting words to a root ("descended" becomes "descend") / dropping common words ("the", "not")
Sentence embedding / encoder	A model that turns text into a vector so that similar meanings land close together

Term	Plain explanation
Frozen	Used as downloaded, with no fine-tuning on our data
Cosine similarity	How closely two vectors point the same way; our similarity score for encoders
Chunk / MaxP	A roughly 150-word piece of a report / a report's score equals its best chunk's score
RRF (reciprocal rank fusion)	Merge two rankings: each report gets the sum of 1 / (60 + rank) from each list
Hybrid	BM25 and an encoder fused with RRF
MRR@10	Average of 1 / rank of the first correct result (0 if not in the top 10). 1.0 = always first
Recall@k	Share of queries whose correct report appears in the top k
P@10	Share of the top 10 that are relevant (grade 2; "lenient" counts grade 1 too)
nDCG@10	Ranking quality from 0 to 1 that rewards highly relevant results near the top
Silver vs. gold labels	Automatic relevance labels from NASA codes vs. human judgments
Pooling	Merging the top 10 of every system into one shuffled sheet so annotators judge blind
Cohen's kappa	Agreement between two annotators beyond chance
Bootstrap CI	Resample queries 10,000 times to get a 95% range for a mean
Paired randomization test	Shuffle which system got which per-query score to see if a difference could be chance (the p-value)
Wilson interval	A confidence interval for an accuracy (used for the contrast test)

Term	Plain explanation
Dev / test split	Dev queries may guide design choices; test queries are only reported
Union-find	A fast algorithm that groups items linked by shared IDs (used for incident grouping)